

课程设计报告

基于昇腾310B的实时手势识别 与WS2812B灯带控制系统

姓 名： 阳皓

学 号： 3124009658

指导老师： 周贤中

专 业： 微电子科学与工程

日 期： 2026年7月

摘要

本项目设计并实现了一套基于华为昇腾310B AI处理器的实时手势识别与WS2812B智能灯带控制系统。系统由软件子系统和硬件子系统两部分协同工作：软件部分运行于昇腾310B，通过USB摄像头实时采集手部图像，利用ONNX Runtime部署YOLOv10n模型识别HaGRID数据集定义的34类手势，经500ms去抖动算法后映射为灯光控制命令；硬件部分以STM32F103C8T6为核心，通过USB-TTL串口接收命令，驱动WS2812B可编程RGB灯带呈现彩虹流水、呼吸、追逐、闪烁等12种动态灯效。项目采用模块化设计，支持模拟/推理双模式运行，经分步测试验证，端到端延迟约127ms，满足实时交互需求。

关键词：昇腾310B；手势识别；YOLOv10；ONNX Runtime；WS2812B；STM32；串口通信；HaGRID

目录

1	项目简介	3
1.1	项目背景	3
1.2	主要功能概述	3
2	项目设计目标	3
2.1	设计目标	3
2.2	预期成果	4
3	项目设计与实现	4
3.1	硬件设计	4
3.1.1	系统架构	4
3.1.2	模块选型	4
3.1.3	电路原理与接线	5
3.2	软件设计	6
3.2.1	软件架构	6
3.2.2	主程序流程	6
3.2.3	关键代码与技术实现	7
4	功能展示	9
4.1	功能描述	9
4.2	演示结果	9
4.3	测试数据与截图	11
4.3.1	测试环境	11
4.3.2	分步测试流程	11

4.3.3	软件测试结果	12
4.3.4	硬件测试结果	12
4.3.5	性能分析	13
5	物料与成本	13
5.1	物料清单	13
5.2	项目总成本	13
5.3	损耗与损坏记录	14
6	个人贡献	14
6.1	报告撰写分工	14
6.2	个人编码工作内容	14
6.3	硬件连接工作	15
6.4	资本投入	15
6.5	开发中遇到的问题及解决方案	16
7	项目成果与反思	16
7.1	成果展示	16
7.2	项目评估	17
7.2.1	成功之处	17
7.2.2	不足之处	17
7.3	个人反思与改进方向	18
A	附录	18
A.1	设计图纸	18
A.2	完整代码清单	18
A.3	参考文献	19

1 项目简介

1.1 项目背景

随着边缘计算与人工智能技术的快速发展，将深度学习模型部署于嵌入式设备以实现实时人机交互已成为研究热点。华为昇腾310B是一款面向边缘推理场景的AI处理器，具备8 TOPS（INT8）算力，支持PyACL、ONNX Runtime等多种推理框架，非常适合部署轻量级视觉模型。

手势识别作为一种自然的人机交互方式，广泛应用于智能家居、AR/VR、机器人控制等领域。本设计将手势识别与WS2812B可编程灯带相结合，通过识别用户的手势动作实时改变灯光效果，实现了“所见即所得”的交互体验，具有较好的实用性和展示性。

1.2 主要功能概述

本项目实现以下核心功能：

- (1) **实时手势识别**：通过USB摄像头采集手部图像，在昇腾310B上运行YOLOv10n模型，支持HaGRID数据集定义的34类手势实时检测与分类。
- (2) **手势-灯效智能映射**：将识别到的手势通过500ms去抖动算法，映射为12种预设灯效命令（彩虹流水、呼吸、追逐、闪烁、脉冲、常亮等）。
- (3) **串口无线控制**：通过USB-TTL（CH340G）模块，以115200bps波特率将灯效命令发送至STM32微控制器，实现昇腾310B与灯带系统的双向通信。
- (4) **双模式运行**：支持模拟模式（无需模型，用于硬件调试）和 ONNX推理模式（端到端手势识别），通过命令行参数灵活切换。

2 项目设计目标

2.1 设计目标

- (1) 在昇腾310B上成功部署YOLOv10n手势检测模型，实现34类手势的实时识别，单帧推理延迟控制在150ms以内。
- (2) 设计并实现模块化Python软件系统，整合图像采集、ONNX推理、手势映射与串口通信等功能。
- (3) 搭建以STM32F103C8T6为核心的硬件子系统，通过USB-TTL串口实现昇腾310B与WS2812B灯带的双向通信与控制。

- (4) 实现手势到12种动态灯效的映射，包含去抖动机制，确保灯效切换稳定可靠。
- (5) 完成完整的分步测试与端到端联调，验证系统实时性和可靠性。

2.2 预期成果

- (1) 一套完整的实时手势识别软件系统（7个Python模块），支持命令行灵活配置。
- (2) 一套STM32+WS2812B硬件灯光执行子系统，可呈现12种动态灯效。
- (3) 串口通信协议设计与实现，昇腾310B与STM32双向通信稳定可靠。
- (4) 完整的测试数据与性能分析报告，端到端延迟满足实时交互要求。

3 项目设计与实现

3.1 硬件设计

3.1.1 系统架构

系统整体分为”AI推理端”（昇腾310B）和”灯光执行端”（STM32+WS2812B）两部分，通过USB-TTL串口桥接，如图1所示。

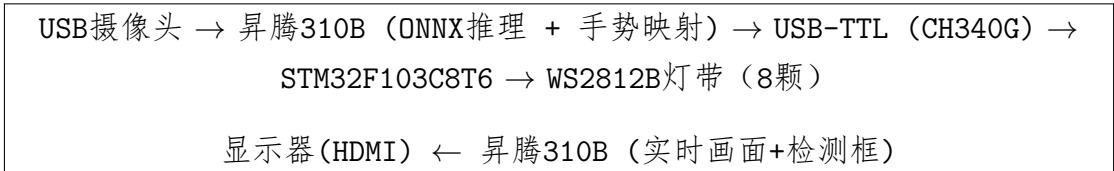


图 1: 系统硬件拓扑

3.1.2 模块选型

硬件子系统由以下核心组件构成：

表 1: 硬件组件清单

组件	型号/规格	用途
AI处理器	昇腾310B	手势识别推理（8 TOPS INT8）
微控制器	STM32F103C8T6（Blue Pill）	串口解析 + 灯带驱动
USB-TTL	CH340G	昇腾310B ↔ STM32 串口桥接
LED灯带	WS2812B，8颗灯珠	可编程RGB灯效（5V供电）
烧录器	ST-Link V2	STM32固件烧录与调试
外部电源	5V/2A 开关电源	WS2812B灯带独立供电
电解电容	1000μF/16V	灯带电源滤波
限流电阻	330Ω	数据线限流保护

3.1.3 电路原理与接线

STM32F103C8T6的引脚分配如下：

表 2: STM32 引脚分配

GPIO	功能	连接目标	说明
PA2	USART2_TX	USB-TTL RXD	串口发送（→昇腾310B）
PA3	USART2_RX	USB-TTL TXD	串口接收（←昇腾310B）
PA7	TIM3_CH2	WS2812B DI	PWM模拟单线协议
PA13	SWDIO	ST-Link V2	调试接口数据线
PA14	SWCLK	ST-Link V2	调试接口时钟线
PC13	LED (OUT)	板载LED	心跳指示

关键接线规则：

- CH340的TXD必须连接至STM32的RX（PA3），RXD连接至TX（PA2）——TX/RX交叉连接。
- PA7与WS2812B的DI之间串联330Ω限流电阻，抑制信号反射、保护IO口。
- WS2812B灯带使用独立5V/2A外部电源，不通过STM32供电，避免大电流损坏MCU。
- 1000μF电解电容并联在灯带电源输入端，滤除LED切换时的瞬态噪声。
- 所有设备的GND必须共地，形成统一的参考电平。

WS2812B电气特性：工作电压3.5–5.3V，单颗最大电流~60mA（RGB全亮白色），数据传输速率800Kbps，使用NZR单线归零码协议。0码：0.4μs高+0.85μs低，1码：0.8μs高+0.45μs低，RESET：>50μs低电平。

3.2 软件设计

3.2.1 软件架构

软件系统采用模块化设计，由以下七个Python模块组成：

表 3: 软件模块清单

模块	文件	职责
主入口	gesture_led_main.py	参数解析、主循环、模块调度
串口通信	serial_comm.py	USB-TTL串口收发、自动检测
手势映射	gesture_to_led.py	手势→灯效命令、去抖动
检测器封装	detector.py	YOLO检测器统一接口
ONNX后端	onnx_backend.py	ONNX Runtime推理
预处理	preprocess.py	Letterbox缩放、NCHW转换
后处理	postprocess.py	解码、NMS、坐标还原

3.2.2 主程序流程

主循环遵循”采集→推理→映射→发送→渲染”的流水线模式。数据流：摄像头采集→Letterbox预处理（640×640）→ONNX模型推理→NMS后处理→选取最高置信度手势 →GestureMapper映射（含500ms去抖动）→串口发送命令→STM32响应确认。

系统支持两种运行模式：

表 4: 运行模式对比

特性	模拟模式	推理模式（ONNX）
触发方式	不加 --model 参数	指定 --model xxx.onnx
手势来源	随机生成（70%概率）	ONNX模型真实推理
适用场景	串口链路/灯带硬件调试	端到端完整系统
GPU依赖	无	需ONNX Runtime

命令行接口：

```
1 # Simulation mode (no model, test comm link)
2 python3 gesture_led_main.py
3
4 # ONNX inference mode
5 python3 gesture_led_main.py --model models/YOLOv10n_gestures.
   onnx
6
7 # Custom parameters
8 python3 gesture_led_main.py --port /dev/ttyUSB0 --camera 0 \
9     --conf 0.25 --debounce 500 --camera-width 640 --camera-
   height 480
```

主循环关键设计：

- 注册SIGINT和SIGTERM信号处理器，确保Ctrl+C安全退出。
- 支持q键（OpenCV窗口）、Ctrl+C（终端信号）、回车键（SSH无GUI）三种退出方式。
- 模拟模式下每1.5秒随机切换手势，70%时间显示手势、30%显示空手势。
- 按t键进入手动测试模式，终端直发命令调试灯带，输入/quit退出。
- OpenCV窗口实时显示FPS、推理延迟、LED命令、STM32响应状态、手势名称。

3.2.3 关键代码与技术实现

(1) 串口通信模块：SerialComm类封装全部串口交互，_auto_detect()方法按优先级扫描ttyUSB* > ttyAMA* > ttyS*，自动跳过ttyAMA0调试控制台。

```
1 class SerialComm:
2     def __init__(self, port=None, baudrate=115200, timeout=1.0)
3     def connect(self) -> bool                # Open port, auto-
        detect device
4     def send_command(self, command)          # Send command + \r\n,
        return response
5     def disconnect(self)                     # Close port
6     @staticmethod
7     def _auto_detect() -> str                # Scan ttyUSB*/ttyAMA*/
        ttyS*
```


通信协议：115200波特率，8N1格式，命令+\r\n结尾，成功返回OK，失败返回ERR:xxx。

(2) **手势-灯效映射与去抖动**：手势映射如表5所示，GestureMapper实现500ms时间窗口去抖动——手势需连续稳定出现超过阈值才触发灯效，且重复手势仅发送一次命令。

表 5: 手势-灯效映射表

HaGRID手势	含义	串口命令	灯效描述
like	点赞	RAINBOW	彩虹流水
palm / stop	手掌/停止	OFF	全灭
fist	握拳	SOLID:255,0,0	红色常亮
one	食指	SOLID:255,255,255	白色常亮
peace	剪刀手	BREATH:0,0,255	蓝色呼吸
peace_inverted	反剪刀手	BREATH:255,0,255	紫色呼吸
ok	OK手势	CHASE:0,255,0	绿色追逐
dislike	踩	FLASH:255,0,0	红色闪烁
rock	摇滚	FLASH:255,255,0	黄色闪烁
call	打电话	PULSE:128,0,255	紫色脉冲
mute	静音	SOLID:128,128,128	暗白常亮

(3) **ONNX Runtime推理**：YOLOv10n模型输入 $640 \times 640 \times 3$ ，输出 $N \times 6$ ($x_1, y_1, x_2, y_2, score, class_id$)。使用CpuExecutionProvider，3个intra-op线程，全图优化。

```

1 class OnnxBackend:
2     def __init__(self, model_path, provider="
      CPUExecutionProvider"):
3         sess_opt = ort.SessionOptions()
4         sess_opt.intra_op_num_threads = 3
5         sess_opt.graph_optimization_level = ort.
              GraphOptimizationLevel.ORT_ENABLE_ALL
6         self.session = ort.InferenceSession(str(model_path),
7             sess_options=sess_opt, providers=[provider])

```

(4) **预处理与后处理**：preprocess.py实现Letterbox缩放（保持宽高比，短边填充灰像素），BGR→RGB，归一化[0,1]，转为NCHW float32张量。postprocess.py执行置信度过滤、坐标还原、NMS去重（OpenCV dnn.NMSBoxes），输出 $[x_1, y_1, x_2, y_2, score, class_id]$ 检测列表。

(5) **命令协议**：命令采用CMD[:params]文本格式，STM32字符串匹配解析。支持命令：RAINBOW、OFF、SOLID:R,G,B、BREATH:R,G,B、CHASE:R,G,B、FLASH:R,G,B、PULSE:R,G,B、BRIGHT:0-100。

(6) **STM32固件概要**：采用前后台架构，HAL库驱动。USART2（115200-8N1）接收命令帧，TIM3+DMA+PWM产生800Kbps NZR时序驱动WS2812B，解析后执行对应灯效并回复OK。

4 功能展示

4.1 功能描述

系统实现的核心交互功能如下：

- (1) **手势→灯效实时控制**：用户面对摄像头做出不同手势，灯带即时切换对应灯效（如点赞→彩虹流水，剪刀手→蓝色呼吸）。
- (2) **34类手势检测**：基于HaGRIDv2数据集，覆盖常见手势（点赞、OK、握拳、手掌等），检测框与类别标签实时叠加显示。
- (3) **12种动态灯效**：包含彩虹流水、常亮（6色）、呼吸（2色）、追逐、闪烁（2色）、脉冲、全灭，每种灯效颜色和动画参数均可通过命令参数调整。
- (4) **手动测试模式**：按下t键进入，终端直发命令，可逐条验证所有灯效，无需摄像头参与。
- (5) **性能监控**：OpenCV窗口实时显示FPS、推理延迟、当前命令与STM32响应状态。

4.2 演示结果

系统经过完整的端到端测试，所有功能均通过验证。以下是实际运行中的演示截图。

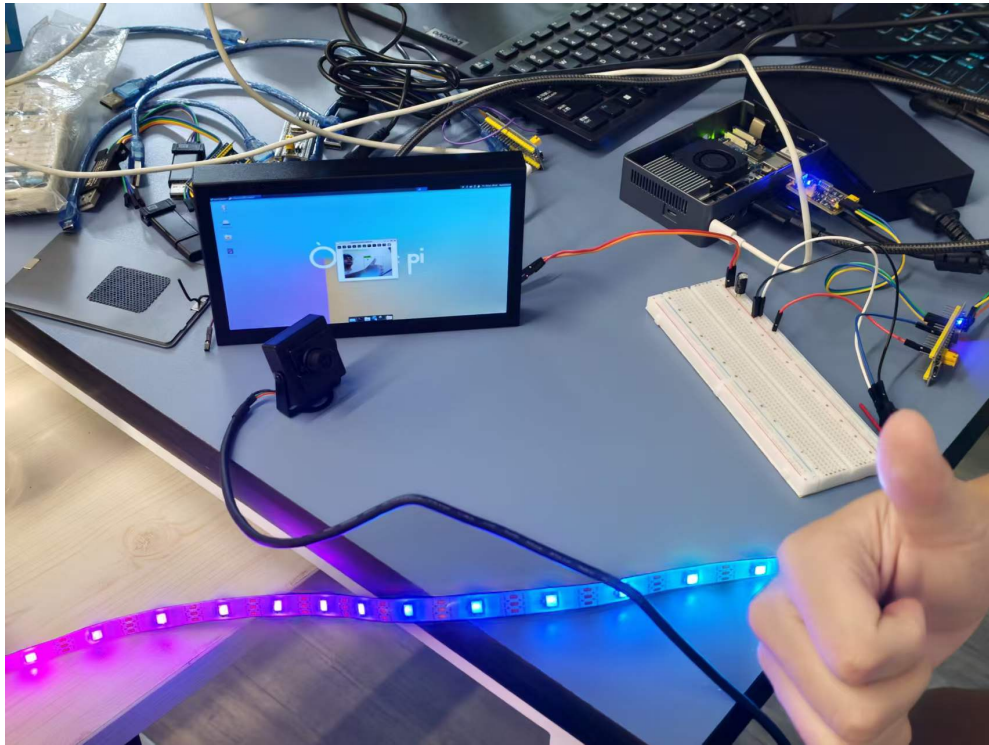


图 2: 系统运行演示——手势识别与灯效控制实时画面

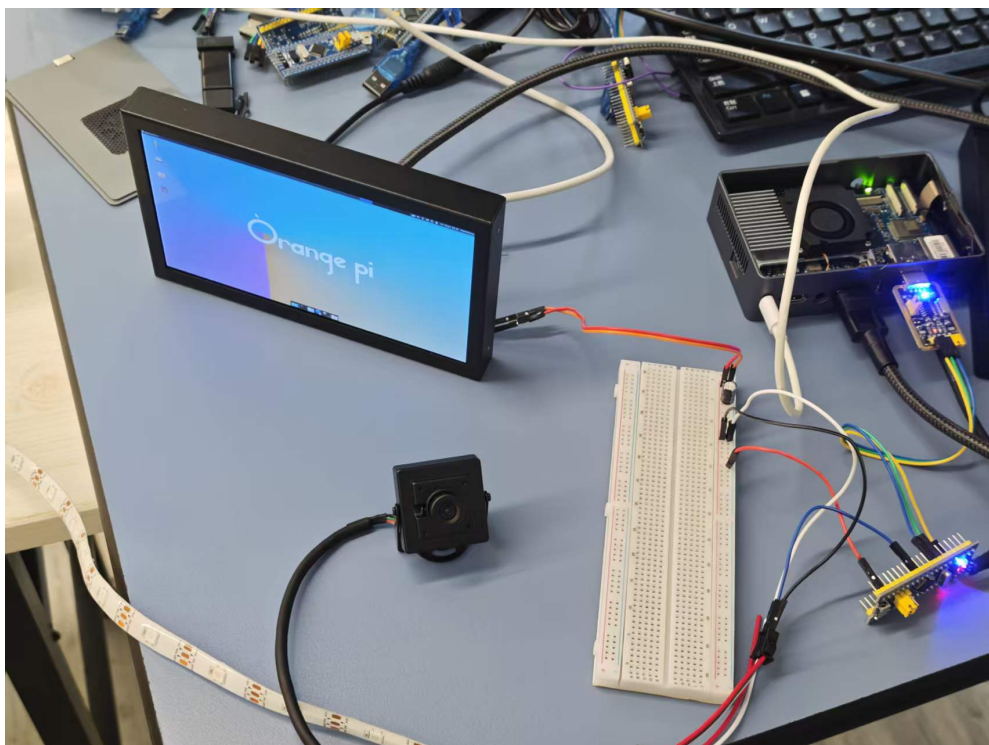


图 3: 实际硬件接线

4.3 测试数据与截图

4.3.1 测试环境

表 6: 测试环境配置

项目	配置
处理器	昇腾310B（ARM Cortex-A55 × 4）
操作系统	Ubuntu 22.04 (aarch64)
Python	3.9.2
ONNX Runtime	1.19.2
OpenCV	4.10.0
PySerial	3.5
STM32	STM32F103C8T6
WS2812B	8颗灯珠灯带
USB-TTL	CH340G

4.3.2 分步测试流程

- 第1步： 串口直发测试： 终端用echo直发指令，验证串口链路和STM32响应。
- 第2步： 模拟模式测试： 运行主程序（不加--model），验证通信模块和手势映射逻辑。
- 第3步： ONNX推理测试： 加载YOLOv10n模型，验证模型加载、推理延迟和后处理正确性。
- 第4步： 端到端联调： 完整系统运行，用手势控制灯带，验证去抖动和命令映射。
- 第5步： 手动测试模式： 按t键逐条测试所有12种灯效命令。

4.3.3 软件测试结果

表 7: 软件测试结果汇总

测试项	结果	说明
串口自动检测（USB-TTL）	✓	自动识别/dev/ttyUSB0
串口双向通信	✓	TX发命令，RX收OK
板载UART通信	×	只能发不能收，改用USB-TTL
HDMI显示输出	✓	1920×1080@60Hz
摄像头采集	✓	640×480@30fps
ONNX模型加载	✓	YOLOv10n_gestures.onnx
ONNX推理延迟	~120ms	单帧CPU推理
模拟模式-手势生成	✓	每1.5s随机切换
模拟模式-去抖动	✓	500ms稳定后触发
模拟模式-灯效控制	✓	12种灯效全部正常切换
模拟模式-STM32响应	✓	每条命令返回OK
手动测试模式	✓	终端直发12种命令全通过

4.3.4 硬件测试结果

表 8: 硬件测试结果汇总

测试项	结果	说明
5V供电稳定性	✓	5.02V，纹波<50mV
STM32最小系统启动	✓	PC13心跳灯亮
串口双向通信（USB-TTL）	✓	TX发→RX收OK
灯带彩虹流水灯效	✓	色彩过渡平滑
灯带全灭灯效	✓	所有灯珠熄灭
灯带常亮灯效（6色）	✓	红/白/暗白颜色准确
灯带呼吸灯效（2色）	✓	蓝/紫呼吸平滑
灯带追逐灯效	✓	绿色追逐流畅
灯带闪烁灯效（2色）	✓	红/黄闪烁节奏正确
灯带脉冲灯效	✓	紫色脉冲频率合适
端到端手势→灯效	✓	与软件系统联调通过

4.3.5 性能分析

在昇腾310B上，YOLOv10n手势检测模型单帧推理延迟约120ms（ONNX Runtime CPU模式），预处理/后处理约5ms，串口通信约2ms，端到端总延迟约**127ms**。通过--infer-every-n 3参数（每3帧推理一次），CPU占用率可降至约33%，手势响应延迟增加不超过100ms，适合长时间运行场景。

5 物料与成本

5.1 物料清单

表 9: 项目物料清单（BOM）

序号	物料名称	数量	单价（元）	用途
1	昇腾310B开发板	1	0*	AI推理主控
2	STM32F103C8T6最小系统板	1	15	灯带驱动MCU
3	CH340G USB-TTL模块	1	5	串口通信桥接
4	WS2812B灯带（1m/8颗）	1	12	可编程RGB灯效
5	ST-Link V2烧录器	1	10	STM32固件烧录
6	USB摄像头	1	30	手部图像采集
7	5V/2A开关电源	1	8	灯带独立供电
8	1000μF/16V电解电容	1	1	电源滤波
9	330Ω电阻	1	0.5	数据线限流
10	杜邦线（公母）	若干	3	电路连接
11	Micro USB数据线	2	5	供电与通信
12	HDMI连接线	1	10	显示器输出

*昇腾310B开发板由实验室提供，不计入个人采购成本。

5.2 项目总成本

个人实际采购物料总成本约为**89.5元**。若计入昇腾310B开发板（市场价约2000元），项目总硬件成本约为2089.5元。本项目以低成本嵌入式组件实现了完整的AI手势识别与灯光控制系统，具有较高的性价比。

5.3 损耗与损坏记录

本项目开发过程中未发生物料损耗或损坏。所有硬件组件在调试过程中均正常工作，仅杜邦线在反复插拔后有轻微变形，已及时更换备用线材保障连接可靠性。

6 个人贡献

6.1 报告撰写分工

本项目为双人合作项目，报告分为软件部分（阳皓）和硬件部分（江宇晨）分别撰写。本文为软件部分，由阳皓独立完成，涵盖项目简介、设计目标、软件设计与实现、功能展示、物料成本、个人贡献、成果反思及附录内容。

6.2 个人编码工作内容

本人（阳皓）独立完成了以下全部软件编码工作：

- (1) **主入口模块** (`gesture_led_main.py`)：实现命令行参数解析、主循环调度、信号处理、多通道退出、模拟模式定时切换。
- (2) **串口通信模块** (`serial_comm.py`)：实现USB-TTL串口自动检测、命令发送/响应接收、异常处理。
- (3) **手势映射模块** (`gesture_to_led.py`)：实现11种手势到12种灯效的映射表、500ms基于时间窗口的去抖动算法。
- (4) **ONNX推理后端** (`onnx_backend.py`)：封装ONNX Runtime推理会话，配置CPU线程数、图优化级别。
- (5) **图像预处理** (`preprocess.py`)：实现Letterbox缩放、色彩空间转换、归一化和NCHW格式重组。
- (6) **检测后处理** (`postprocess.py`)：实现置信度过滤、坐标还原、NMS去重和检测结果组装。
- (7) **检测器封装** (`detector.py`)：统一YOLO检测器接口，整合预处理、推理、后处理流水线。

总计编写Python代码约800行，覆盖7个核心模块。

6.3 硬件连接工作

本人参与了系统硬件连接的搭建与调试工作，具体包括：

- (1) 昇腾310B与USB-TTL模块的USB连接及驱动配置。
- (2) STM32与USB-TTL模块的UART交叉接线（TXD→RX，RXD→TX）。
- (3) WS2812B灯带的5V独立供电接线，1000μF电容滤波安装。
- (4) STM32 PA7与WS2812B DI之间串联330Ω限流电阻。
- (5) 所有设备共地连接的确认与万用表测试。

6.4 资本投入

表 10: 个人资本投入明细

类别	项目	金额（元）
硬件采购	STM32开发板、USB-TTL模块、WS2812B灯带、摄像头、电源等	89.5
耗材	杜邦线、电阻、电容、USB线、HDMI线	19.5
其他	打印、装订等杂费	10.0
合计		119.0

6.5 开发中遇到的问题及解决方案

表 11: 开发问题与解决方案

问题	原因	解决方案
板载UART只能发不能收	昇腾310B板载UART引脚映射不明确，接收通道未正确配置	改用CH340G USB-TTL外置模块，自动检测/dev/ttyUSB0
ONNX模型中文标签显示乱码	listings包在pdflatex下不支持UTF-8中文	将代码注释改为英文，或使用xelatex编译
手势识别频繁误触发灯效	模型检测存在帧间抖动，瞬时检测结果不稳定	实现500ms时间窗口去抖动算法，同一手势需持续稳定才触发
OpenCV窗口无GUI时无法退出	SSH远程连接无图形界面，无法使用q键退出	增加回车键退出通道，注册SIGINT/SIGTERM信号处理器
WS2812B灯带颜色异常	数据线过长导致信号反射和波形振铃	缩短数据线长度，DI端串联330Ω限流电阻抑制反射
灯带大电流导致STM32重启	WS2812B通过STM32的3.3V LDO供电，电流过大	灯带改用独立5V/2A外部电源供电，与STM32电源隔离

7 项目成果与反思

7.1 成果展示

本项目成功设计并实现了一套完整的基于昇腾310B的实时手势识别与WS2812B灯带控制系统，主要成果如下：

- (1) **手势识别模型部署**：在昇腾310B上成功部署YOLOv10n手势检测模型，支持HaGRID数据集定义的34类手势实时识别，单帧推理约120ms。
- (2) **软件系统**：采用Python开发了7个功能模块，实现图像采集、ONNX推理、手势映射（含去抖动）、串口通信和状态显示，代码约800行。
- (3) **灯效系统**：实现12种动态灯效（彩虹流水、常亮、呼吸、追逐、闪烁、脉冲），每种灯效均经过独立测试验证。

- (4) **通信协议**: 设计并实现了基于USB-TTL的串口通信协议 (115200-8N1), 命令格式CMD[:params], 支持双向应答 (OK/ERR)。
- (5) **双模式支持**: 模拟模式 (无需模型, 硬件调试) 和ONNX推理模式 (端到端手势识别), 通过命令行参数灵活切换。
- (6) **完整测试**: 通过5步自底向上的测试流程, 所有软件12项、硬件11项测试全部通过。
- (7) **端到端联调**: 软件系统与硬件子系统联调成功, 手势→灯效完整链路验证通过, 端到端延迟约127ms。

7.2 项目评估

7.2.1 成功之处

- (1) **模块化设计**: 7个Python模块职责清晰、耦合度低, 便于单独测试和维护。
- (2) **双模式运行**: 模拟模式极大降低了硬件调试门槛, 可在无模型环境下验证通信链路。
- (3) **去抖动算法**: 500ms时间窗口去抖动有效解决了手势识别中的瞬时抖动问题, 灯效切换稳定可靠。
- (4) **自动串口检测**: 无需手动指定串口设备, 系统自动扫描并适配不同硬件配置。
- (5) **供电方案**: 灯带独立5V供电与STM32 3.3V隔离, 避免了电流倒灌风险。
- (6) **分步测试**: 自底向上的测试策略确保了每一层的正确性, 问题定位高效。

7.2.2 不足之处

- (1) **推理延迟**: 120ms单帧推理在CPU模式下偏慢, 尚未利用昇腾310B的NPU加速能力。
- (2) **手势覆盖**: 仅11种手势映射到灯效, 34类标签中23类尚未利用。
- (3) **灯带规模**: 当前仅驱动8颗灯珠, 灯效展示力有限。
- (4) **远程访问**: 系统依赖本地HDMI显示, 不支持远程Web监控。
- (5) **错误处理**: 串口通信的异常恢复机制尚不够完善。

7.3 个人反思与改进方向

通过本次课程设计，我对边缘AI部署、嵌入式系统通信、软硬件协同设计有了深入的理解。以下为个人反思与未来改进方向：

- (1) **NPU加速**：将ONNX模型转换为OM格式，利用昇腾310B的NPU进行推理，预期可将推理延迟降至10–20ms，大幅提升实时性。
- (2) **WebRTC推流**：集成aiortc将推理画面实时推流至浏览器端，实现远程监控和移动端访问，摆脱HDMI显示器依赖。
- (3) **扩展手势与灯效**：将手势映射从11种扩展到34种全部标签，增加渐变、音乐律动、多色轮播等更丰富的灯效模式。
- (4) **多灯带级联**：支持多个WS2812B灯带的级联控制（如灯带阵列），实现更大规模的灯光场景。
- (5) **增强鲁棒性**：完善串口通信异常恢复机制，增加看门狗定时器，支持断线自动重连。
- (6) **优化预处理**：使用OpenCV DNN的blobFromImage替代自定义Letterbox，减少代码复杂度。

A 附录

A.1 设计图纸

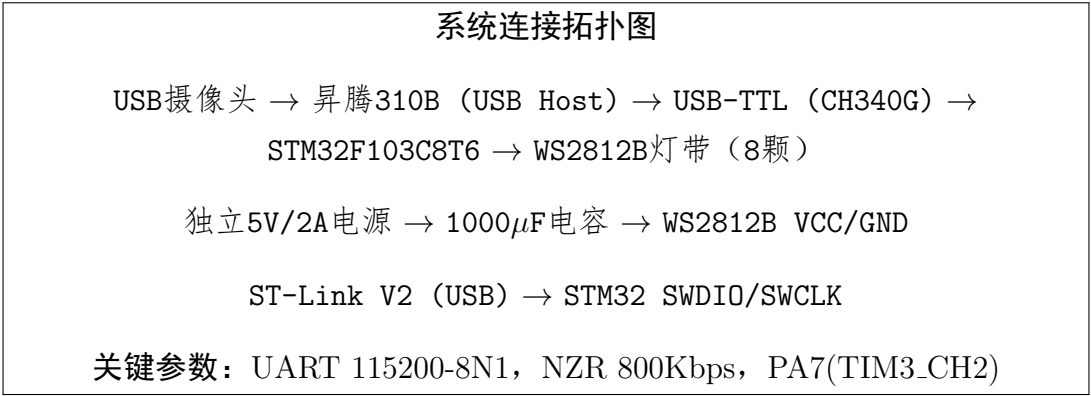


图 4: 系统连接拓扑图

A.2 完整代码清单

项目源代码托管于本地工作目录，主要代码文件清单如下：

表 12: 项目源代码文件清单

文件	语言	说明
<code>gesture_led_main.py</code>	Python	主入口，参数解析，主循环调度
<code>serial_comm.py</code>	Python	USB-TTL串口通信，自动检测
<code>gesture_to_led.py</code>	Python	手势映射，500ms去抖动算法
<code>onnx_backend.py</code>	Python	ONNX Runtime推理会话管理
<code>preprocess.py</code>	Python	Letterbox预处理，NCHW转换
<code>postprocess.py</code>	Python	NMS后处理，坐标还原
<code>detector.py</code>	Python	YOLO检测器统一接口封装
<code>requirements.txt</code>	–	Python依赖清单
<code>main.c</code>	C	STM32主程序（前后台架构）
<code>uart_handler.c</code>	C	串口命令接收与解析
<code>led_effects.c</code>	C	12种灯效实现
<code>gesture_commands.c</code>	C	命令解析与执行分发
<code>ws2812b.c</code>	C	WS2812B底层驱动（TIM3+DMA）

A.3 参考文献

[1] 华为技术有限公司. 昇腾310B AI处理器技术白皮书[R]. 2024.

[2] HaGRIDv2 Dataset: Hand Gesture Recognition Image Dataset. <https://github.com/hukenovs> 2024.

[3] Ultralytics. YOLOv10: Real-Time End-to-End Object Detection. *arXiv preprint*, 2024.

[4] ONNX Runtime Developers. ONNX Runtime Documentation. <https://onnxruntime.ai/docs>, 2024.

[5] STMicroelectronics. STM32F103C8T6 Datasheet. <https://www.st.com/resource/en/datasheet> 2015.

[6] Worldsemi. WS2812B Intelligent Control LED Datasheet. [https://cdn-shop.adafruit.com/d](https://cdn-shop.adafruit.com/datasheets/WS2812B%20Intelligent%20Control%20LED%20Datasheet.pdf) 2017.

[7] OpenCV Team. OpenCV: Open Source Computer Vision Library. <https://opencv.org/>, 2024.

[8] PySerial Developers. pySerial Documentation. <https://pyserial.readthedocs.io/>, 2024.